



Computer Engineering Dept.
Faculty of Engineering
Cairo University



Compiler Project

Phase 1

NAME	Code	Sec	BN
Abdelrahman Mostafa Gomaa	9220475	1	31
Mohammed Khaled Ahmed	9220681	2	48
Salma Mohammed Entsar	9220364	1	24
Youssef Roshdy Agaiby	9220985	2	63



Computer Engineering Dept.
Faculty of Engineering
Cairo University



Overview

A compiler for a subset of C++ Programming language using lex and yacc, The tokens are matched in the lex file and then passed to the parser to see if these tokens can form a valid statement supported in our language.

Tools & Technology

- Lex
- Yacc
- C
- simple Makefile

Lexer

Language

Data Types

- int
- float
- string
- char
- bool

Variables

variable names are a sequence of characters from [A-Za-z0-9] starting with [A-Za-z] and its value can be of type string, float, int, char and bool.



Operations

- =, ||, &&
- |, ^, &
- >=, <=, ==, !=, >, <
- <<, >>
- +, -
- *, /, %
- !, -
- ++, --

Tokens

INT_TYPE	matches the “int” keyword used for declaration.
FLOAT_TYPE	matches the “float” keyword used for declaration.
CHAR_TYPE	matches the “char” keyword used for declaration.
VOID_TYPE	matches the “void” keyword used for declaration.
BOOL_TYPE	matches the “bool” keyword used for declaration.
STRING_TYPE	matches the “string” keyword used for declaration.
IF	matches the “if” keyword and returns it as a token to the parser.
ELSE	matches the “else” keyword and returns it as a token to the parser.
WHILE	matches the “while” keyword and returns it as a token to the parser.
DO	matches the “do” keyword and returns it as a token to the parser.
FOR	matches the “for” keyword and returns it as a token to the parser.
BREAK	matches the “break” keyword and returns it as a token to the parser.
CONTINUE	matches the “continue” keyword and returns it as a token to the parser.
RETURN	matches the “return” keyword and returns it as a token to the parser.
SWITCH	matches the “switch” keyword and returns it as a token to the parser.



CASE	matches the “case” keyword and returns it as a token to the parser.
DEFAULT	matches the “default” keyword and returns it as a token to the parser.
CONST	matches the “case” keyword for variables and returns it as a token to the parser.
LEFT_SHIFT	recognize the “<” operator.
RIGHT_SHIFT	recognize the “>” operator.
INC	recognize the “++” operator.
DEC	recognize the “--” operator.
EQ	recognize the “==” relational operator.
NE	recognize the “!=” relational operator.
GE	recognize the “>=” relational operator.
LE	recognize the “<=” relational operator.
AND	recognize the “&&” logical operator.
OR	recognize the “ ” logical operator.
NOT	recognize the “!” logical operator.
ASSIGN	recognize the “=” operator.
GT	recognize the “>” relational operator.
LT	recognize the “<” relational operator.
BITWISE_AND	recognize the “&” bitwise operator.
BITWISE_OR	recognize the “ ” bitwise operator.
BITWISE_XOR	recognize the “^” bitwise operator.
ADD	recognize the “+” operator.
SUB	recognize the “-” operator.
BITWISE_NOT	recognize the “~” bitwise operator.
MUL	recognize the “*” operator.
DIV	recognize the “/” operator.
MOD	recognize the “%” operator.



SEMICOLON	recognize “;”, default behavior of semicolons in c++.
PRINT	for printing in the console takes the expression in two parentheses ‘()’.
BOOLEAN	handles boolean values can be “true” or “false”.
VARIABLE	handles variable and function names.
CHARACTER	handles character values (can be any single character).
STRING	handles string values.
INTEGER	handles integer values.
FLOAT	handles float values.

Parser

our program can be a sequence of function declarations or variable declarations, and in this phase it handles the syntax analysis.

Functions

functions take the form `type VARIABLE '(' parameters ')' statement`

valid examples:

- `int main() { // statement_list }`
- `int sum(int x, int y =1);`

Variables

variables declarations take the form

`( optional const keyword ) type var_declarement SEMICOLON;`

valid examples:

- `int x =1;`
- `int x;`
- `int x,y=1;`
- `const int x =1,y;`



Computer Engineering Dept.
Faculty of Engineering
Cairo University



Control Flow statements

For loop:

valid examples:

- `for(;;);`
- `for(int i =0;i<5;i++){//statement}`
- `for(i =2;;);`

while loop:

valid examples:

- `while(1);`
- `while(i < 6) {//statement}`

Do while loop:

valid examples:

- `do a++; while(1);`
- `do {//statement} while(a<5);`

Switch case:

valid example:

```
switch (x) {  
  case 1:  
    print("One\n");  
    break;  
  default:  
    print("Other\n");  
}
```

If statement:



Computer Engineering Dept.
Faculty of Engineering
Cairo University



valid examples:

- `if(a==1) a++;`
- `if(a==1) a++; else a--;`

Break and Continue:

valid examples:

- `if(a==1)break;`
- `if(a==1)continue;`

How to run

first write the program in test.bas file and simply run the following command **Make run**.

Thank you for reading.